UNIVERSIDAD DE CARABOBO
FACULTAD DE CIENCIAS Y TECNOLOGÍA
DEPARTAMENTO DE COMPUTACIÓN
ARQUITECTURA DEL COMPUTADOR

PRÁCTICA DE LABORATORIO #4

Elaborado por:

Jesús Torres — C.I. 31.022.489

Christian Báez — C.I. 31.473.355

Fecha:

7 de octubre de 2026

Índice

1. Ciclo de una Interrupción de Hardware 2

2. Gestión de E/S: Sondeo vs. Interrupciones 2

3. Ventajas de las Interrupciones en el Procesador 3

4. Registros Especiales en MIPS32 3

5. Importancia de Guardar el Contexto 4

6. Generación de Excepciones en MIPS32 4

7. Estrategias de Tratamiento de Excepciones e Interrupciones 4

8. Habilitación de Interrupciones 5

9. Procesamiento de Interrupciones 5

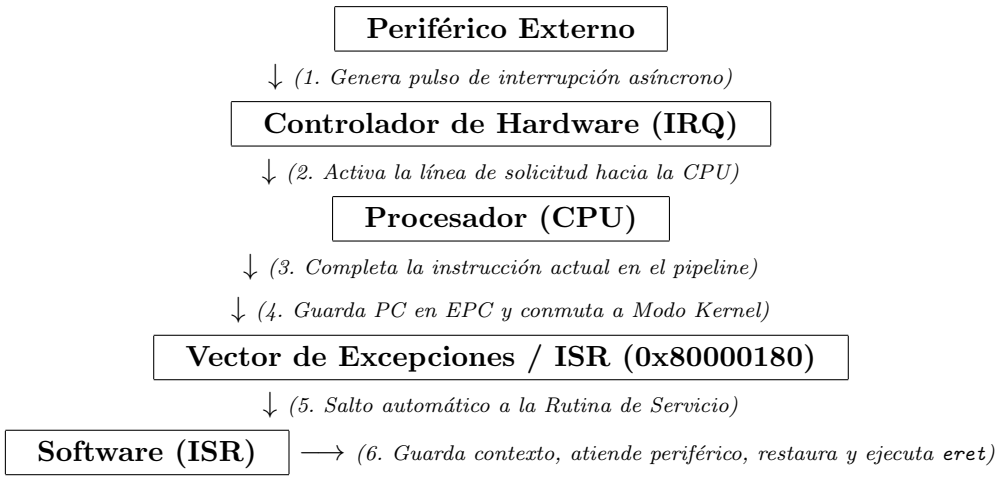
10. Interrupciones de Reloj y Control de Ejecución 5

11. Análisis y Discusión de los Resultados 6

1. Ciclo de una Interrupción de Hardware

1. Explique con un diagrama cómo funciona el ciclo de una interrupción de hardware (desde que ocurre el evento hasta que se atiende).

A continuación se presenta el diagrama esquemático que ilustra el flujo y ciclo completo de atención de una interrupción de hardware:



Descripción detallada del ciclo:

- **1. Ocurrencia:** Un dispositivo periférico (como un teclado o un temporizador) genera un evento físico o lógico y activa su señal de interrupción asíncrona.
- **2. Solicitud:** La señal viaja por una línea física de hardware hacia el controlador de interrupciones del sistema (IRQ).
- **3. Finalización de instrucción:** El procesador completa por completo la ejecución de la instrucción actual que se encuentra en curso dentro del pipeline.
- **4. Resguardo y Conmutación:** El hardware guarda la dirección de retorno en el registro `EPC` (Exception Program Counter) del Coprocesador 0, conmuta el estado operativo a modo kernel y deshabilita temporalmente las interrupciones globales.
- **5. Salto a la ISR:** El flujo del procesador se redirige obligatoriamente hacia la dirección del vector de excepciones (en MIPS estándar, típicamente `0x80000180`).
- **6. Atención y Retorno:** El software de la Rutina de Servicio de Interrupción (ISR) guarda los registros en la pila, atiende la solicitud del periférico, restaura los registros y ejecuta la instrucción especial `eret` para reanudar el flujo del programa interrumpido.

2. Gestión de E/S: Sondeo vs. Interrupciones

2. ¿Qué diferencias hay entre gestionar E/S con sondeo y hacerlo con interrupciones?

- **Sondeo (*Polling*):** El procesador se encarga de consultar de manera activa, cíclica y constante el registro de estado de un periférico para determinar si hay datos listos. Esto mantiene a la CPU ocupada en una espera pasiva inútil, desperdiciando ciclos de procesamiento valiosos.
- **Interrupciones:** El procesador se dedica a ejecutar tareas útiles. El periférico permanece inactivo hasta que ocurre un evento, momento en el cual envía una señal eléctrica (interrupción) para notificar a la CPU. La CPU interrumpe momentáneamente lo que hace, atiende el evento y retoma su labor anterior con alta eficiencia.

3. Ventajas de las Interrupciones en el Procesador

3. ¿Qué ventajas tiene el uso de interrupciones en términos de uso del procesador?

- Eliminación absoluta de la espera activa, evitando que la CPU desperdicie ciclos preguntando constantemente a dispositivos lentos.
- Mayor eficiencia global, permitiendo que la CPU ejecute procesos útiles de manera continua mientras los dispositivos operan en segundo plano.
- Capacidad de respuesta inmediata en tiempo real ante eventos críticos del entorno.
- Soporte fundamental para la multiprogramación y la multitarea en sistemas operativos modernos.

4. Registros Especiales en MIPS32

4. ¿Qué registros especiales se utilizan en MIPS32 para gestionar interrupciones?

Para la gestión de excepciones e interrupciones, MIPS32 utiliza registros específicos ubicados en el Coprocesador 0 (CP0):

- `Status` (Registro CP0 12): Controla las máscaras de interrupción y la habilitación global del procesador.
- `Cause` (Registro CP0 13): Almacena el código específico que originó la última excepción o interrupción, además de las líneas de hardware pendientes.
- `EPC` (Registro CP0 14): Almacena la dirección virtual de la instrucción interrumpida o afectada.
- `BadVAddr` (Registro CP0 8): Guarda la dirección virtual de memoria que generó un fallo de direccionamiento (*Address Error*).

5. Importancia de Guardar el Contexto

5. ¿Por qué es necesario guardar el contexto (registros) al entrar en una rutina de servicio?

Porque la Rutina de Servicio de Interrupción (ISR) utiliza registros de propósito general (`$t0`, `$v0`, etc.) para realizar sus operaciones y cálculos internos. Si dichos registros no se respaldan previamente en la pila (*stack*), la ISR sobrescribirá los valores que el programa principal estaba utilizando en ese instante, corrompiendo su estado y generando fallos graves o resultados incorrectos al retornar.

6. Generación de Excepciones en MIPS32

6. Momentos en que pueden generarse excepciones en un sistema MIPS32.

a) **Situaciones que generan excepciones:**

1. Desbordamiento aritmético en operaciones con signo.
2. Fallo de dirección de memoria (*Address Error* por accesos mal alineados o prohibidos).
3. Instrucción ilegal o código de operación no reconocido por la arquitectura.
4. Llamadas al sistema o puntos de interrupción mediante instrucciones como `syscall` o `break`.

b) **Etapas del pipeline que pueden provocar excepciones:**

- *IF (Instruction Fetch):* Fallos de dirección o TLB al buscar la instrucción en memoria.
- *ID (Instruction Decode):* Códigos de operación inválidos al decodificar la instrucción.
- *EX (Execution):* Desbordamientos aritméticos detectados por la ALU.
- *MEM (Memory):* Errores de alineación o violaciones de permisos en operaciones de carga (`lw`) o almacenamiento (`sw`).

7. Estrategias de Tratamiento de Excepciones e Interrupciones

7. Estrategias de tratamiento de excepciones e interrupciones.

a) **Diferencias:** Las interrupciones son **asíncronas** (provienen de eventos externos al flujo del programa), mientras que las excepciones son **síncronas** (generadas de forma directa y predecible por la instrucción en ejecución).

b) **Estrategias y redirección:** Se emplean estrategias centralizadas y vectoriales. La ejecución se redirige forzando el Contador de Programa (PC) a la dirección del manejador en modo kernel. El registro `EPC` (*Exception Program Counter*) cumple la función fundamental de almacenar la dirección de retorno exacta para reanudar el flujo mediante la instrucción `eret`.

8. Habilitación de Interrupciones

8. Habilitación de interrupciones en dispositivos y procesador.

a) **Forma de habilitación:**

- *Teclado y Pantalla:* Activando el bit de habilitación de interrupción (*Interrupt Enable*, bit 1) en sus respectivos registros de control MMIO.
- *Procesador:* Modificando el registro `Status` del Coprocesador 0, activando el bit global `IE` (bit 0) y configurando las máscaras de interrupción de hardware correspondientes (bits 8 a 15).

b) **Consecuencia de habilitar solo en dispositivos:** Si se habilitan las interrupciones en los dispositivos pero no en el procesador, las solicitudes de hardware serán ignoradas por completo, comportándose el sistema como si operara exclusivamente por sondeo.

9. Procesamiento de Interrupciones

9. Procesamiento de interrupciones.

a) **Ciclo de una interrupción de reloj:** Ocurre el pulso de reloj → La línea se activa → La CPU termina la instrucción actual → El hardware guarda el PC en `EPC` y salta al vector de interrupción (`0x80000180`) → La rutina salva los registros en la pila → Se procesa el evento del temporizador → Se restauran los registros → Se ejecuta la instrucción de retorno `eret`. El hardware automatiza el salto y resguardo de estados, mientras que el software (sistema operativo/ISR) gestiona la lógica de atención y restauración.

b) **Importancia del contexto:** Guardar el contexto garantiza la transparencia del sistema, asegurando que el programa interrumpido continúe su ejecución de manera intacta y sin pérdida de datos.

10. Interrupciones de Reloj y Control de Ejecución

10. Interrupciones de reloj y control de ejecución.

a) **Usos del reloj:** Permiten implementar el reparto de tiempo (*time-sharing*). Si un programa queda atrapado en un bucle infinito o agota su cuota de tiempo, la interrupción de reloj fuerza el control hacia el sistema operativo para suspenderlo o abortarlo. También actúan como mecanismos de control de tiempo (*timeout*) para finalizar programas que superan su límite de ejecución.

b) **Acción del software al finalizar antes de tiempo:** Debe desarmar o deshabilitar el temporizador activo para evitar interrupciones espurias innecesarias una vez que el proceso ha finalizado.

11. Análisis y Discusión de los Resultados

- **Evaluación del Ejercicio 1 (Buffer Circular y Memoria Mapeada - MMIO):** La implementación del buffer circular de tamaño fijo demostró ser una solución eficiente y elegante para la gestión temporal de flujos de datos asíncronos en MIPS32. Al utilizar los registros de control y datos del simulador MMIO (`0xffff0000` y `0xffff0004`), el programa logró evaluar el estado del teclado de forma continua sin bloquear el ciclo de procesamiento. La aplicación de aritmética modular (`head = (head + 1) % buf_size`) permitió reutilizar de manera indefinida el espacio de almacenamiento asignado de 64 bytes. Asimismo, el mecanismo de filtrado condicional, acotando strictly el rango de códigos ASCII entre 65 y 90 (`A-Z`), garantizó que cualquier dígito, símbolo de puntuación o letra minúscula ingresada fuera descartado de forma transparente antes de alterar el buffer, cumpliendo a cabalidad con las especificaciones del diseño.

- **Evaluación del Ejercicio 2 (Máquina de Estados Temporizada del Semáforo):** La simulación del semáforo controlado por la tecla de entrada `s` evidenció la correcta estructuración lógica de una máquina de estados basada en duraciones fijas (20 segundos de transición, 10 segundos en amarillo y 30 segundos en rojo). Un aspecto crítico resuelto durante la ejecución fue el manejo del tiempo a través del `syscall 30` (milisegundos desde la época). Debido a la gran magnitud de los valores de tiempo devueltos por el simulador MARS, el uso de instrucciones aritméticas y de comparación sin signo (`subu` y `bgeu`) resultó fundamental para prevenir desbordamientos de signo (*sign overflow*), asegurando que los retardos temporales transcurrieran con absoluta precisión cronométrica y sin interrupciones espurias.

- **Implicaciones Prácticas y Arquitectónicas:** Ambos ejercicios permitieron contrastar la teoría del procesamiento de E/S en arquitecturas de computadores. Aunque la aproximación utilizada se basó en sondeo activo (*polling*), el control estricto de intervalos de tiempo y la manipulación directa de estructuras de datos en bajo nivel reflejan cómo interactúan los programas de sistema con los controladores de periféricos. Estos laboratorios reafirman que un dominio preciso de los saltos condicionales, el manejo de punteros en memoria y la gestión de temporizadores son pilares indispensables para el posterior desarrollo de sistemas operativos y rutinas de atención por interrupciones en hardware real.